

Number Systems in Node.js

Overview

A **number system** defines how numbers are represented using symbols (digits).

System	Base	Digits Used	Example
Decimal	10	0–9	245, 123
Binary	2	0, 1	1010
Octal	8	0–7	754
Hexadecimal	16	0–9, A–F	1F4, A2

Decimal Number System (Base 10)

- Most commonly used system.
- Each digit has a place value (powers of 10).
- Example: $435 = 4 \times 10^2 + 3 \times 10^1 + 5 \times 10^0$

Binary Number System (Base 2)

- Used by computers internally (0s and 1s).
- Each bit (binary digit) is either 0 or 1.
- Example: $1011 = 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 = 11$

Convert Binary to Decimal in Node.js:

```
let binary = "1010"; // Also can be stored like this : 0b1010
let decimal = parseInt(binary, 2);
console.log(decimal); // 10
```

Octal Number System (Base 8)

- Uses digits from 0 to 7.
- Common in file permissions in Unix/Linux (e.g., `chmod 755`).

Convert Octal to Decimal in Node.js:

```
let octal = "17"; // Also can be stored like this : 0o17
let decimal = parseInt(octal, 8);
console.log(decimal); // 15
```

Hexadecimal Number System (Base 16)

- Used in memory addresses, colors (**#FF5733**), etc.
- Digits: 0-9 and A-F (A=10, ..., F=15).

Convert Hex to Decimal in Node.js:

```
let hex = "1A"; // Also can be stored like this : 0x1A
let decimal = parseInt(hex, 16);
console.log(decimal); // 26
```

Useful Node.js Conversion Snippets

```
let decimal = 255;

console.log(decimal.toString(2)); // Binary: 11111111
console.log(decimal.toString(8)); // Octal: 377
console.log(decimal.toString(16)); // Hex: ff
```

Digital Data Units

- **1 Bit (b)** = Smallest data (0 or 1)
- **1 Nibble** = 4 bits
- **1 Byte (B)** = 8 bits
- **1 KB** = 1,024 Bytes
- **1 MB** = 1,024 KB
- **1 GB** = 1,024 MB
- **1 TB** = 1,024 GB
- **1 PB** = 1,024 TB
- **1 EB** → **ZB** → **YB** (keeps increasing)

Remember

Bit ≠ **Byte** → 1 Byte = 8 Bits

Binary Data on Physical Level

Binary data (0s and 1s) are not just abstract values—they are physically represented in hardware.

What is Binary?

- Binary is a base-2 number system: only **0** and **1**.
- In computers, **0** = **OFF** and **1** = **ON**.

Physical Representation Methods

1. Electric Signals (in CPUs & RAM)

- Voltage Levels:
 - **High Voltage = 1 (ON)**
 - **Low Voltage = 0 (OFF)**
- Transistors act as switches to store and process binary values.

2. Magnetic Storage (HDDs)

- Hard disks use magnetized surfaces.
 - **North polarity = 1**
 - **South polarity = 0**

3. Optical Media (CD/DVD)

- Data stored as **pits** and **lands**.
 - Pit = 0, Land = 1 (or vice versa depending on reflection)

4. Solid-State Drives (SSDs)

- Store charge in floating gate transistors:
 - **Charged = 1**
 - **No charge = 0**

Data Flow in Computer (Simplified)

1. **Input Device** (e.g., keyboard) sends binary signals.
2. **CPU** processes it using transistors.
3. **RAM/Storage** stores it as voltage/magnetic/charge states.
4. **Output Device** (e.g., screen) converts it back to visible form.

Example Analogy

Concept	Binary	Real-world Analogy
Switch	0/1	Light bulb ON/OFF
Magnetism	0/1	Compass N/S
Voltage	0/1	Battery Full/Empty

Why It Matters

Understanding physical binary helps in:

- Building hardware
- Understanding low-level programming
- Troubleshooting digital systems

Character Sets vs Character Encodings

Character Set (Charset)

- A **collection of characters** (letters, numbers, symbols). Assigning a number to a character.
- It **defines what characters exist**.
- Example sets:
 - **ASCII**: A–Z, a–z, 0–9, basic symbols (128 total)
 - **Unicode**: Covers almost all characters from all languages

Think of a character set as the **alphabet**.

Character Encoding

- A **mapping (link) of characters to bytes** (numbers).
- It **tells the computer how to store and read characters**.
- Example encodings:
 - **UTF-8**: Variable-length encoding for Unicode (most common)
 - **UTF-16**: 2 or 4 bytes per character
 - **ISO-8859-1**: European languages

Think of encoding as the **secret code** to convert characters into binary.

Example

- Character: **A**
- Unicode (charset) assigns it: **U+0041**
- UTF-8 (encoding) stores it as: **01000001** (binary)

Summary Table

Aspect	Character Set	Character Encoding
What it defines	List of characters	How characters are stored in bytes
Example	ASCII, Unicode	UTF-8, UTF-16, ISO-8859-1
Role	What exists	How to save & interpret it

Use in Programming

- Always specify both in files or HTML:

```
<meta charset="UTF-8">
```

UTF-8 (Unicode Transformation Format - 8 bit)

🧠 What is it?

- A **character encoding** used to represent **Unicode** characters.
- Uses **1 to 4 bytes** to store a character.
- Fully compatible with **ASCII** (first 128 characters).

◇ Why UTF-8?

- 🌐 Supports all world languages (emoji, scripts, symbols).
- 💾 Saves space (efficient for English text).
- 🔒 Standard for web (HTML, JSON, XML).

◇ Byte Structure

Byte Length	Character Range	Example
1 byte	U+0000 – U+007F	A, B, 0–9
2 bytes	U+0080 – U+07FF	ñ, ç
3 bytes	U+0800 – U+FFFF	آ, ش
4 bytes	U+10000 – U+10FFFF	👨, 🐉

◇ Example (Encoding 'A')

- A = U+0041
- UTF-8: **01000001** (same as ASCII)

◇ Features

- ☒ Variable-length encoding
- ☒ Backward-compatible with ASCII
- ☒ No byte-order issues (unlike UTF-16)

◇ Common Uses

- 🌐 Websites (`<meta charset="UTF-8">`)
- 💾 JSON, XML, CSV
- 📦 APIs and databases

🌐 Unicode Planes

◇ What are they?

- Unicode organizes characters into **17 planes**.
- Each plane has **65,536 (2¹⁶)** code points.

✈ Planes Overview

Plane No.	Name	Range	Usage Example
0	Basic Multilingual Plane	U+0000–U+FFFF	Common scripts, emojis
1	Supplementary Multilingual	U+10000–U+1FFFF	Rare characters, scripts
2	Supplementary Ideographic	U+20000–U+2FFFF	Chinese, Japanese, Korean
14	Supplementary Special	U+E0000–U+EFFFF	Language tags, variation
15–16	Private Use Areas	U+F0000–U+10FFFF	Custom characters

 Summary

- Unicode has **17 planes (0–16)**.
- Most characters are in **Plane 0 (BMP)**.
- Higher planes store **rare, historical, or custom** characters.

UTF-8 vs UTF-16 vs UTF-32

Feature	UTF-8	UTF-16	UTF-32
 Bytes/Char	1–4 bytes	2 or 4 bytes	Always 4 bytes
 Size	Small for English	Medium	Large
 Use Case	Web, JSON, files	Windows, Java, .NET	Internal processing
 ASCII Support	☒ Same as ASCII	☒ Needs conversion	☒ Needs conversion
 Endianness	☒ Not needed	☒ Required (BOM)	☒ Required (BOM)
 Variable?	☒ Yes	☒ Yes	☒ No (fixed)

 Summary

- **UTF-8**: Best for web, compact, ASCII-friendly.
- **UTF-16**: Balanced, used in Windows/Java.
- **UTF-32**: Simple but uses more space.

Byte Order Mark (BOM) and Endianness

 Endianness

- **What is it?**
The **byte order** in which multibyte data is stored in memory.

◇ Big Endian

- Stores the **most significant byte first** (big-end first).

- Example (0x12345678):
 📦 Memory: 12 34 56 78

◇ Little Endian

- Stores the **least significant byte first** (little-end first).
- Example (0x12345678):
 📦 Memory: 78 56 34 12

📌 Byte Order Mark (BOM)

- **What is it?**
A special invisible character (U+FEFF) at the **start of a text file** to indicate encoding (mainly in UTF formats) and the **endianness** (byte order) of the file.
- **Why used?**
 - ☒ Tells the software which Unicode encoding is used (UTF-8, UTF-16, etc.)
 - ☒ Helps handle **endianness** in UTF-16/UTF-32
- **Examples: BOM in Different Encodings**
 - UTF-16 LE (Little Endian) BOM: FF FE
 - UTF-16 BE (Big Endian) BOM: FE FF
 - UTF-32 LE (Little Endian)**: FF FE 00 00.
 - UTF-32 BE (Big Endian)**: 00 00 FE FF.

✗ **UTF-8 doesn't require BOM**, the BOM is optional and is not strictly needed because UTF-8 is byte-oriented and does not have endianness issues. but some editors still add it. However, when present, the BOM in UTF-8 is represented by the byte sequence EF BB BF.

⚠ BOM and Endianness

- In UTF-16/UTF-32:
 - BOM helps determine **endianness**.
 - Without BOM, software must **guess** the byte order.